



Instituto Politécnico Nacional

Escuela Superior de Cómputo

Selected Topics in Cryptography

Reporte Práctica 05:

“ECDH with standard elliptic curves”

Alumno:

Sigala Morales Said

2022630413

Grupo: 7CM4

Maestro: Sandra Diaz Santiago

Fecha de realización: 02 de abril de 2025

Fecha de entrega: 02 de abril de 2025



PROGRAMMING EXERCISES

Instituto Politécnico Nacional

Escuela Superior de Cómputo

Selected Topics in Cryptography



Design and implement a function to simulate key agreement using ECDH scheme between two entities. Your implementation must fulfill the following requirements

```
1 def ecdh_key_agreement(curve_name, save_to_file, filename):
2
3     if curve_name == "P-224":
4         curve = ec.SECP224R1()
5     elif curve_name == "P-256":
6         curve = ec.SECP256R1()
7     elif curve_name == "P-384":
8         curve = ec.SECP384R1()
9     elif curve_name == "P-521":
10        curve = ec.SECP521R1()
11    else:
12        raise ValueError("Curva elíptica no válida.")
13
14
15    private_key = ec.generate_private_key(curve, default_backend())
16    public_key = private_key.public_key()
17
18    if save_to_file:
19        pem = public_key.public_bytes(
20            encoding=serialization.Encoding.PEM,
21            format=serialization.PublicFormat.SubjectPublicKeyInfo
22        )
23        with open(filename, "wb") as f:
24            f.write(pem)
25        print(f"Clave pública guardada en {filename}")
26    else:
27        pem = public_key.public_bytes(
28            encoding=serialization.Encoding.PEM,
29            format=serialization.PublicFormat.SubjectPublicKeyInfo
30        )
31        print("Clave pública (Base64):", base64.b64encode(pem).decode())
32
33    other_private_key = ec.generate_private_key(curve, default_backend())
34    other_public_key = other_private_key.public_key()
35
36    shared_key = private_key.exchange(ec.ECDH(), other_public_key)
37
38    derived_key = hashlib.sha256(shared_key).digest()
39
40    print("Clave derivada (Base64):", base64.b64encode(derived_key).decode())
41
42    return derived_key
```



Descripción de la función:

Primero se elige una "curva elíptica". Después, cada uno (Alice y Bob) genera un par de números secretos: uno que guardan para sí mismos (la clave privada) y otro que comparten con el otro (la clave pública).

Alicia usa su combinación secreta y la llave pública de Beto para crear un nuevo número secreto. Beto hace lo mismo, usando su combinación secreta y la llave pública de Alicia. A pesar de que nunca compartieron sus combinaciones secretas, ambos terminan con el mismo número secreto.

Este número secreto es como una contraseña que solo ellos conocen. Para hacerlo aún más seguro, lo mezclamos un poco con una función (SHA-256), que lo convierte en una clave muy difícil de adivinar.

Finalmente, convertimos esta clave en un formato llamado Base64, que es como un código que se puede leer en internet. Y listo, Alicia y Beto tienen una clave secreta que pueden usar para cifrar sus mensajes, ¡y nadie más puede descifrarlos!